

Validation

Spring 2026

Introduction

- In backend development, one of the most critical responsibilities of an API is controlling and validating incoming data.
- Every request sent to an Express server contains data coming from external clients, browsers, mobile applications, or even malicious actors.
- Without proper validation:
 - invalid data may enter the system
 - database consistency may be corrupted
 - security vulnerabilities may appear
 - application crashes may occur
 - business rules may be violated
- **Validation is not optional.**
- It is a fundamental layer of backend architecture.

Validation

- Validation is the process of checking whether incoming data satisfies predefined rules before the application processes it.
- Typical validation checks:
 - Required fields
 - Data type checks
 - Length constraints
 - Format verification
 - Range limitations
 - Business rule enforcement

Example

- A user registration endpoint may require:
- Validation **ensures** these are satisfied before continuing.

Field	Rules
name	required, minimum 3 chars
email	valid email format
password	minimum 8 chars
age	must be positive integer

Why important?

- Security: Protection against malicious input
 - SQL Injection
 - XSS
 - malformed JSON
 - oversized payloads
 - unexpected object structures

Data Integrity

- Data Integrity: Applications rely on consistent and reliable data.
 - e.g. Age cannot be < 0 .
- Poor data quality leads to
 - Broken analytics
 - Failed business logic
 - Corrupted reports
 - Unreliable systems
- Validation guarantees data quality before persistence.

Preventing Application Errors

- Unexpected input frequently causes runtime exceptions.

Business Rule Enforcement

- Validation is not only technical
- It also enforces domain rules.
- Examples
 - stock cannot be negative
 - appointment date cannot be in the past
 - username must be unique
 - order quantity must be greater than zero.
- These protect business consistency.

Better API Design

- Well validated APIs provide predictable behavior.
- Clients receive
 - consistent error responses
 - meaningful messages
 - clear expectations
- This improves developer experience significantly.

Types of Validation

- Client-side Validation
- Server-side Validation

Client-Side Validation

- Performed in frontend applications
 - HTML form validation
 - required, type=number etc.
 - Better UX
 - Immediate feedback
 - Reduced server load
-
- **IT IS NEVER ENOUGH**
 - Users can bypass frontend controls entirely.

Server-Side Validation

- Performed in the backend.
- The authoritative validation layer.
- Every application must validate incoming requests server-side regardless of frontend validation.

What should be validated?

- req.body → POST / PUT payloads
- req.params → route parameters
- req.query → query strings
- req.headers → authorization headers
- uploaded files → file type, size

Common Validation Rules

- Required Validation
 - Field must exist
- Type validation
 - Must be string / number / boolean etc.
 - e.g. age should be a number
- Length validation
 - min/max char count
 - password ≥ 8
- Format Validation
 - Checks formatting (email, URL, UUID, phone number)
- Range Validation
 - Must be between x and y
- Enum Validation
 - Restricts values to predefined options

Sanitization vs Validation

- Validation
 - Checks whether data is acceptable
 - → Is email valid?
- Sanitization
 - Modifies or cleans data
 - Trimming spaces
 - Escaping HTML
 - Converting to lowercase
 - Removing dangerous characters etc.

Flow

Client Request



Validation Layer



Sanitization



Business Logic



Database

Validation in Express

- Can do manually but it would be repetitive and hard to maintain
- Solution: **Middleware**
- There are popular libraries
 - express-validator (Express middleware validation)
 - Joi (schema validation)
 - Yup (sv)
 - Zod (type-safe validation)
 - ...
-

Common Mistakes

- Trusting Frontend Validation
 - Never trust client-side validation alone
- Missing nested object validation
- Returning Poor Error Messages
 - You should give some information but make sure its not private information

Conclusion

- Validation is one of the core pillars of backend engineering.
 - Improves security
 - Protects databases
 - Enforces business rules
 - Prevents runtime failures
 - Increases API reliability
 - Improves maintainability

Joi

- Schema-based validation library.
- Main idea
 - Define a schema → validate incoming data against that schema
- Creates
 - Centralized validation
 - Reusable rules
 - Readable Code
 - Scalable architecture

Example

- Suppose we want to validate a JSON with **username** and **age** fields.
- We create a JOI schemas.

Basic required schema

```
const Joi = require("joi")

const schema = Joi.object({
  username: Joi.string().required(),
  age: Joi.number().required()
})
```

Required fields

```
const schema = Joi.object({  
  email: Joi.string().required()  
})
```

Min / Max Length

```
const schema = Joi.object({  
  username: Joi.string()  
    .min(3)  
    .max(20)  
    .required()  
})
```


Email Validation

```
const schema = Joi.object({  
  email: Joi.string()  
    .email()  
    .required()  
})
```

Number Validation

```
const schema = Joi.object({  
  age: Joi.number()  
    .min(18)  
    .max(100)  
})
```

Array Validation

```
const schema = Joi.object({  
  tags: Joi.array()  
    .items(Joi.string())  
})
```

Nested object validation

```
{  
  "user": {  
    "name": "John",  
    "age": 30  
  }  
}
```

```
const schema = Joi.object({  
  user: Joi.object({  
    name: Joi.string().required(),  
    age: Joi.number().required()  
  })  
})
```

Enum validation

```
role: Joi.string()  
      .valid("user", "admin", "moderator")
```

Default Values

```
const schema = Joi.object({  
  role: Joi.string()  
    .default("user")  
})
```

Custom Error Messages

```
const schema = Joi.object({  
  password: Joi.string()  
    .min(8)  
    .required()  
    .messages({  
      "string.min": "Password must be at least 8 characters",  
      "any.required": "Password is required"  
    })  
})
```

```
app.use(express.json())

app.post("/users", (req, res) => {

  const schema = Joi.object({
    username: Joi.string().min(3).required(),
    email: Joi.string().email().required(),
    age: Joi.number().min(18)
  })

  const { error, value } = schema.validate(req.body)

  if (error) {
    return res.status(400).json({
      success: false,
      message: error.details[0].message
    })
  }

  res.json({
    success: true,
    data: value
  })
})

app.listen(3000)
```

```
{
  "username": "john123",
  "email": "john@example.com",
  "age": 25
}
```

```
{
  "username": "jo",
  "email": "invalid"
}
```


Sanitization Features

- Joi can also transform data.
- Trim strings
 - name: `Joi.string().trim()`
- convert to lowercase
 - email: `Joi.string().lowercase()`

Reusable Validation Schemas

- Can have [user.schema.js](#) like structure.
- Later you can create a [validate.js](#) and add it to your route easily.

```
const Joi = require("joi")

const createUserSchema = Joi.object({
  username: Joi.string().min(3).required(),
  email: Joi.string().email().required(),
  password: Joi.string().min(8).required()
})

module.exports = {
  createUserSchema
}
```

validate.js

```
const validate = (schema) => {  
  
  return (req, res, next) => {  
  
    const { error } = schema.validate(req.body)  
  
    if (error) {  
      return res.status(400).json({  
        success: false,  
        errors: error.details  
      })  
    }  
  
    next()  
  }  
}  
  
module.exports = validate
```

```
app.post(  
  "/users",  
  validate(createUserSchema),  
  controller  
)
```

